When Faster Code Does Not Mean Faster Delivery: Continuity Asymmetry in AI-Assisted Software Development at Dell Technologies

David Molamphy
Dell Technologies
Limerick, Ireland
david.molamphy@dell.com

Brian Fitzgerald
Lero - University of Limerick
Limerick, Ireland
brian.fitzgerald@ul.ie

Kieran Conboy
Lero - University of Galway
Galway, Ireland
kieran.conboy@universityofgalway.ie

***Abstract*—Large-scale organizations are rapidly adopting AI-assisted development tools, often accompanied by claims of dramatic gains in software delivery performance. Yet software engineering extends beyond code production, and such claims may understate the organizational complexities that shape delivery outcomes. We report a two-phase qualitative case study at Dell Technologies examining the impact of AI-assisted development practices. Alongside practitioner accounts of unprecedented advances in code generation, internal repository and project data indicate a four-fold increase in pull-request activity and a corresponding rise in completed stories.**

However, respondents consistently describe organizational coordination, decision-making, and validation as emerging constraints. We characterize this phenomenon as continuity asymmetry: implementation activities appear to accelerate substantially, while upstream and downstream processes continue at the established organizational cadences. Echoing how DevOps blurred the boundary between development and operations, practitioner accounts suggest that AI-accelerated implementation may require closer integration between product management and development, aligning with the longstanding Continuous Software Development agenda. To support this, we introduce *four continuity gaps* that practitioners can use to identify and address constraints within their own delivery systems.

***Index Terms*—continuous software development, large-scale agile, AI-assisted development, socio-technical systems**

I. Introduction

Business leaders and engineering practitioners are being told — by industry commentators, software vendors and others — that AI-assisted development now enables them to build software faster than ever. Specific to code generation, this claim has substance. AI development tools generate accepted code contributions at meaningful rates [1], and enterprise-scale deployments of generative AI report a substantial impact on developer efficiency [2] with studies reporting development tasks being completed 55.8% faster with AI assistance [3]. Adoption of these tools is similarly broad and fast, with the work of coding assistants identified in an estimated 22–29% of sampled GitHub projects within months of their emergence [4], although evidence from open-source repositories suggests the resulting velocity gains may prove transient, with significant concerns relating to the potential impact on quality and verification [5], [6]. However, we must remember that the generation of code is only one component within what is an inherently socio-technical endeavor [7], and in large-scale organizations, an array of constraining factors beyond development activity alone must be considered [8].

Large-scale software organizations have long been constrained less by the speed of writing code than by the structures surrounding it. Sub-optimal organizational structures, a lack of alignment [9], and dependencies between software teams, support functions and external entities [10] represent failings in socio-technical joint optimization [11]. We propose that AI has rapidly reconfigured the local socio-technical arrangement around implementation, without an equivalent redesign of adjacent socio-technical functions. Regardless of whether these factors have been exacerbated, or made more obvious by AI-assisted development, our central argument is that Continuous Software Development (CSD) is the correct organizing logic for this new era — and the socio-technical factors it addresses are more critical now than ever.

In making this argument, we ask:

RQ1. When AI-assisted development accelerates implementation in a large-scale enterprise, where does the binding constraint on software delivery lie, and through what socio-technical mechanisms does it bind?

RQ2. What organizational adaptations do practitioners consider necessary to restore continuity between accelerated implementation and the wider delivery system, and to what extent do these align with Continuous Software Development as an organizing logic?

Both questions presuppose that implementation has accelerated, which we treat as a premise supported by the literature, practitioner accounts, and internal repository data (Section V-A), rather than as a finding.

Contributions. (1) Enterprise-scale qualitative accounts written from inside a global technology organization on AI-assisted development and the socio-technical mechanisms of the surrounding delivery system. (2) *Continuity asymmetry*: an empirically grounded construct, corroborated by internal project data, describing the mismatch between AI-accelerated implementation and the organizational cadence at which intent

is formed, work is coordinated, output is assured, and decisions are made. (3) A continuity-gap diagnostic that links the frictions we observe to CSD, offering practitioners a means of identifying which constraint binds in their own organization (Section VI-C).

II. Background and Related Work

A. Continuous Software Development

Continuous Software Development (CSD) is a concept encompassing agile, lean, and DevOps practices [12], [13]. CSD promotes rapid and iterative software delivery by focusing on alignment of business strategy and the broader organization beyond technology teams into a continuous flow of value-creating activities [14], [15], enabling organizations to respond to customer demands in a continuous, evolving flow of delivery through effective feedback loops [9]. The need for CSD is driven, as Fitzgerald and Stol argue, by the "harmful disconnects" between major activities from which software development suffers. Notably, the authors framed the business-facing extension of continuity by direct analogy with DevOps: "the recent emphasis on DevOps recognizes that the integration between software development and its operational deployment needs to be a continuous one. We argue a similar continuity is required between business strategy and development, **BizDev** being the term we coin for this" [13]. Continuity, in this view, must extend beyond software development teams across the *broader organization* [15], [13]. Adopting continuity at scale, however, is an organization-wide transformation rather than an isolated change for development teams. Klotins and Gorschek conclude that "the main hindrances to continuous flow of software arise from sub-optimal organizational structures and the lack of alignment", with automation in development practices valuable only once those are removed [9]; such transformations can generate organizational conflicts among stakeholders with differing objectives [8], and organizations differ in readiness [16]. Further, Tkalich et al. find that dependencies *between software teams and support functions, and between software teams and external entities* are the primary sources of delays and bottlenecks in large-scale CSD [10]; Barbala et al. demonstrate that CSD transformation entails three interconnected dimensions — organizational, platform, and cultural [17] —, establishing CSD as a multi-dimensional organizational capability.

B. Large-Scale Agile and Its Frictions

Although "large-scale" lacks a universally agreed definition, Edison, Wang and Conboy (2022) note that prior work characterizes it as "multiple stakeholders and complex projects within large organizations" [18], criteria well aligned to our case study. Despite its prevalence, large-scale agile implementation is complex, with challenges including the fundamental assumption of small, co-located teams (Dingsøyr et al. 2018) [19] and collaboration across other teams and business units [20]. Frameworks such as Scrum-at-Scale, SAFe, and others deliver mixed results [18], and are typically confined to software teams, introducing disconnects between development and other business units [20], at odds with the multi-dimensional construct of CSD. Bosch and Holmström Olsson's (2024) claim that we are witnessing the "The end of agile as we know it" [21] reaffirms that agile principles, formed in small-team, bespoke-software contexts, may necessitate a successor paradigm for the AI era.

C. AI-Assisted and Agentic Development: Adoption and Its Limits

AI assistance now spans the software development life-cycle, from requirements analysis and design to code generation, testing, review, and maintenance [22], and has evolved rapidly from code completion toward agentic systems that navigate repositories, execute commands, and generate and run tests — increasingly positioned not solely as a productivity tool, but as a means of augmenting multiple stages of organizational workflows [23]. Studies of AI coding-assistant users indicate strong perceptions of improved productivity [24], with 84% of participants in one study reporting positive changes to their daily working practices [25], alongside persistent concerns regarding the trustworthiness, correctness, and contextual suitability of AI-generated output [1], [25], [24]. Evidence indicates that industrial deployment of LLM-based code review does not necessarily translate into faster end-to-end delivery [26], a phenomenon this study expands on.

Repository-level evidence on realized acceleration is similarly nuanced. AI-assisted commits are larger than those authored by humans alone [4], [1]. He et al. find a "statistically significant, large, but transient increase in project-level development velocity" accompanied by persistent increases in static analysis warnings and code complexity, identifying quality assurance as "a major bottleneck" for adopters [5], whilst Agarwal et al. find velocity gains largely confined to projects without prior AI tooling, suggesting "diminishing returns to AI assistance" [6]. Alvanakis Apostolou et al. conclude that "output verifiability is the primary enabler of agentic adoption": later life-cycle phases, evaluable through executable feedback, are the most mature, whereas requirements analysis, design and architecture, and project management remain the least explored [27]. Taken together, these studies suggest the benefits of acceleration are bounded by the capacity to specify and assure what is generated — a boundary which is as much organizational as it is technical.

D. AI in Scaled Agile and Continuous Software Development

Research at the specific intersection of AI-assisted development and scaled agile or CSD remains sparse. In a review of AI-driven assistants within SAFe, Saklamaeva and Pavlič identify 18 challenges organizations confront when implementing scaled agile methods, together with seven benefits and five challenges of incorporating AI [28]. The scaled-agile challenges they identify encompass "coordination, resource utilization, and resistance to change" — for the most part, challenges of coordinating work rather than producing it, whilst AI assistance has since advanced most rapidly in production.

Bosch and Holmström Olsson make a related observation at the organizational level, finding "few examples of companies that use AI to reinvent and fundamentally change their existing practices"; as one of their interviewees summarizes, "We automate separate tasks, but we don't reinvent the end-to-end engineering cycle" [29]. A picture emerges in which AI is introduced into scaled organizations at a task level, whilst the surrounding operating model remains largely unchanged. What remains missing is empirical evidence of what happens when AI-driven acceleration meets traditional development methodologies at scale — evidence which this study aims to contribute.

E. Socio-Technical Systems as an Analytical Lens

Socio-Technical Systems Theory (STS) begins from the proposition that organizational effectiveness cannot be explained or improved by viewing technology as a construct standing apart from the social mechanisms within which it is embedded. Appelbaum (1997) states that organizations are effectively combinations of social and technical parts that are open to their environments and argues that effective design depends on 'joint optimization' rather than the traditional pattern of designing the technical system first and then fitting people to it [11]. Per *Conway's Law*, organizational fragmentation, amplified in large organizations, is not merely a coordination problem: communication boundaries and distributed decision rights become embedded in architectures, interfaces and ownership structures, reproducing organizational discontinuities within the technical system itself [30]. Despite the widespread adoption of agile methods in industry, the socio-technical challenges of software development are an often-neglected topic of research across the domain (Elazhary et al. 2022) [31]. In this study we adopt the socio-technical hexagon by Davis et al. [32] — Goals, People, Processes/Procedures, Culture, Technology, Buildings/Infrastructure — not merely as a coding taxonomy but as an analytical lens. This is crucial as, in the specific context of CSD, the concept of *joint optimization* is core to the proposition of a continuous flow of the development machinery across the entire organization, consistent with the holistic nature of the BizDev concept [13].

III. Industrial Context and Definitions

A. Definitions and the Boundary of Our Claims

Terminology in this field is not yet consolidated [33], so we define the practices present in our setting and attempt to bound our claims accordingly.

AI-assisted development: We define this as the use by developers of generative-AI tools (IDE-integrated, chat, and agentic systems, e.g., GitHub Copilot/Claude Code/Devin-class tooling) to generate, modify, explain, review, and document code, with the developer initiating and integrating the work. This corresponds to the *assistive* and early *integrated* stages of Mandl and Mandl's transformation model, in which AI use progresses from informal, individual assistance toward workflows embedded in processes and tooling, under governance and verification mechanisms [33].

Specification-Driven Development (SDD): Specification-Driven Development is not a new concept. Ostroff et al. (2004) outlined SDD as a combination of Test-Driven Development (TDD) and a "specification-first approach" to software requirements. The approach has long recognized the importance of detailed specifications at the start of the development process [34]; however, in the age of AI-assisted development, the practice has witnessed a resurgence. As Piskala (2026) states, "AI models are excellent at pattern completion but poor at mind reading" [35]. In leveraging AI assistants, clear requirements and specifications are critical in achieving valid and consistent output.

B. Organizational Context

Dell Technologies is one of the world's largest technology companies, with approximately 100,000 employees globally, providing hardware and software products from consumers through to governments and Fortune 500 enterprises. The study population spans two major business units and covers software domains from web and cloud platforms, APIs and on-the-box client software, to embedded systems. Whilst the organization embraces AI tooling, the surrounding operating model remains a formally structured, large-scale organization with interlocked team dependencies, centralized DevOps and platform functions, and staged governance (e.g., review board approvals etc.). A number of initiatives are underway in the organization at the time of writing to transition toward an AI-native Specification-Driven Development methodology, discussed in Section VI-D.

IV. Methodology

A. Study Design

We conducted a qualitative single-case study in two sequential phases. **Phase 1** analyzes open-text responses from three organization-wide "snapshot" surveys. These surveys are conducted at defined intervals throughout the year across the in-scope business units at Dell Technologies with a view to understanding the challenges and factors impacting team members directly involved in software delivery. **Phase 2** comprises twelve semi-structured interviews with practitioners across the delivery system. Interviews were designed to specifically probe the mechanisms identified in Phase 1. Descriptive data from existing internal reporting augment the qualitative findings: weekly PR/MR volume with median lines of code from repositories, and monthly story completion and epic status histories from Jira across 25 projects. For epics, we compared only closed epics' time in pre-development, the one phase permitting a like-for-like comparison without distortion from epics still in progress or recently created (hence the Jan-Sep '25 and '26 periods). A case study suits a contemporary, rapidly evolving phenomenon studied in its natural organizational context; our design and reporting follow Runeson and Höst's guidelines [36]. Ethical approval for the study was granted by the University of Limerick with the approval of Dell Technologies. Due to Dell-internal data confidentiality requirements, our raw data cannot be shared.

B. Phase 1 Data: Survey Datasets

The surveys were administered to 528 software developers across the two business units in September 2025, February 2026, and July 2026, eliciting open-ended commentary attached to developer-experience topics which we leverage to establish the broader organizational context and its evolution over time. Whilst the surveys are predominantly quantitative (these data are excluded from our analysis), we leveraged the optional, open-text fields across the survey sections (perspective, culture, workflow and tooling) for this purpose. The average survey response rate was 83%; however, many responses contained no open-text comments. Of the 921 comments captured (Wave 1: 248, Wave 2: 268, Wave 3: 405), 881 (96%) were valid and codable.

TABLE I
INTERVIEW PARTICIPANTS

ID	Role	ID	Role
P1	Lead SW Architect	P7	Principal SW Engineer
P2	Sr. SW Eng. Manager	P8	Sr. SW Engineer
P3	Director, SW Eng.	P9	Sr. Product Manager
P4	Customer Validation Lead	P10	Sr. SW Eng. Manager
P5	Executive Leader, SW Eng.	P11	Sr. SW Engineer
P6	Mid-Level SW Engineer	P12	Sr. Director, SW Eng.

C. Phase 2 Data: Practitioner Interviews

Interviews were conducted between July and October 2026. Practitioners were purposively sampled based on role and project across the software development system in order to span the full context of a complex large-scale environment. Table I summarizes participants and their roles.

These semi-structured interviews, conducted via Microsoft Teams, with audio-to-text transcription used to produce transcripts for subsequent analysis, typically spanned 30–60 minutes. One senior leader (P12) was deliberately interviewed last, after preliminary themes had been derived, and was shown the emerging continuity-gap model; this interview therefore also served as practitioner feedback on the model (Section VI-C). Quotations are presented verbatim (ellipses mark omitted text), with the exception of correction of obvious transcription errors and redaction of individual, team and product names and other confidential data. Across both the survey and interview datasets, participant anonymity is protected by ID codes and generalized job titles.

D. Data Analysis: Codebook-Structured Thematic Analysis

Methodological position. We analyze both datasets using **codebook thematic analysis**, in which codes are the smallest unit of analysis and the foundation for themes organized around central concepts [37], [38], [39]. The codebook was derived from a predefined theoretical frame, the socio-technical hexagon [32], whose six components form the top-level codes. In both phases, provisional coding was performed by an LLM (Anthropic's Claude Opus 5.5) [40] in a restricted environment under strict author-defined rules, with positive and negative examples provided for each researcher-defined code. All coding decisions (AI and human) were documented; 73.8% of initial AI codings were accepted, the balance being rejected and modified. The researchers retained interpretive responsibility throughout, and thematic development and synthesis were entirely author-led. We disclose this process in line with ACM guidelines on the use of AI in the research lifecycle [41].

TABLE II
MOST PREVALENT SUB-COMPONENT CODES IN THE DATASET.

Hexagon component	Sub-component code	n
Processes/Procedures	Cross-team coordination & dependencies	113
Processes/Procedures	Documentation practices	109
Technology	Developer PCs/devices & IT restrictions	107
Technology	CI/CD & build infra reliability	64
People	Workload & focus	54
Technology	Test environments & hardware	49
Technology	Build & verification latency	48
Processes/Procedures	Planning & requirements	47
Processes/Procedures	Decision-making & governance	45
Technology	AI tool capability & cost	45
Processes/Procedures	Incremental delivery & batch size	38
Processes/Procedures	Code review workflow	37
Processes/Procedures	Meeting & interruption load	33
Processes/Procedures	Human oversight of AI output	32
Technology	AI code quality & trust	32

Phase 1 process. Deductively, each comment was classified against the six hexagon components. Inductively, an initial review of recurring issues produced a provisional sub-component code list per component, consolidated by merging overlapping labels and writing explicit boundary rules (e.g., pipeline *failure* $\rightarrow$ Technology; pipeline *design* $\rightarrow$ Processes/Procedures). Each comment was coded against at least one sub-component (multi-coding permitted for distinct issues). The first author confirmed, edited, or rejected each AI coding suggestion, carrying forward only confirmed or edited codings; a secondary reviewer (an experienced researcher within the organization) then reviewed these decisions and reassessed ambiguous cases. Forty-three distinct sub-component codes were applied across the 881 codable comments.

Phase 2 process. Interview transcripts were coded against the same codebook, using the same AI-assisted coding and two-stage human review. Interview themes were derived consistently with Phase 1, and a cross-instrument synthesis then combined both phases into an integrated dataset.

V. FINDINGS

We first establish the acceleration premise on which both RQs rest (Section V-A), then organize our findings by RQ, with both data collection phases treated as a synthesized dataset. Before discussing these findings, however, Table II provides an overview of the socio-technical components and sub-components most prevalent in the dataset. We present these not as a quantitative measure of frequency, but as a qualitative indicator of issues referenced by respondents.

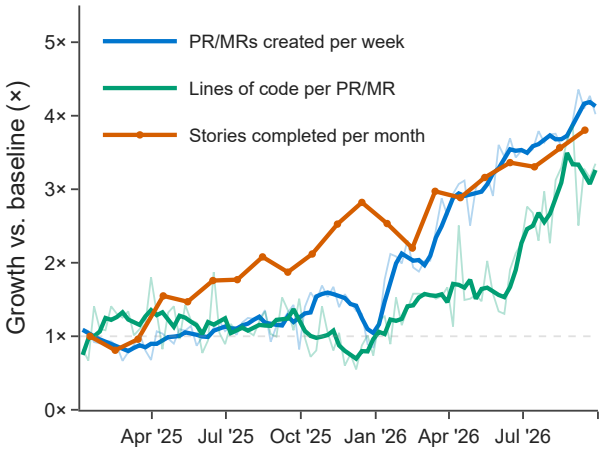


Fig. 1. Weekly PR/MRs created and lines of code per PR/MR, and monthly stories completed, January 2025 to October 2026.

A. The Acceleration Premise: Real Capability, Conditional Trust

Our RQs presuppose that implementation has accelerated. We support this premise using internal repository data across the two business units in our case study. Between January 2025 and October 2026, the number of pull/merge requests (PR/MRs) created per week rose approximately 4.2-fold, whilst median lines of code per PR/MR increased approximately 3.1-fold, with almost all of this growth occurring during 2026 (Fig. 1). Over the same period, the number of stories completed per month rose approximately 3.8-fold, though this growth began earlier, during 2025. More code is therefore entering review, validation and deployment, and in larger units, a combination practitioners associate with greater verification effort (DX0010; Section V-B3). Across both phases, practitioners describe AI assistance as a real and improving capability rather than a speculative one; by the July 2026 wave, AI assistants had become embedded in daily routines for many respondents: "it's a powerful tool that I've been using everyday and it has been helping me to overcome issues and speedup deliveries" (DX0048). The trust practitioners extend to these tools, however, is conditional rather than general, and the condition most frequently articulated concerns the gap between volume and precision: "Generating a lot of mediocre code is easy, generating targeted high quality code is something we are still striving to achieve" (DX0004). Trust in the tools is also intertwined with trust in their inputs: "a slippery slope towards slop... like sloppy specifications and sloppy implementations of them" (DX0906). Capability, in other words, is acknowledged and increasingly relied upon, but is bounded by the effort required to make its output dependable.

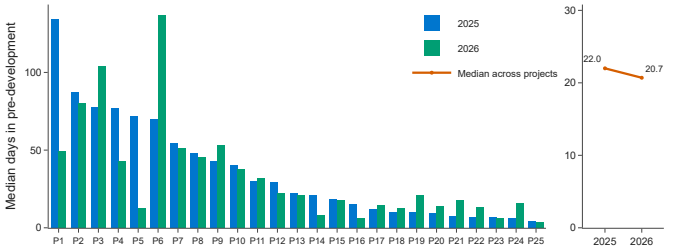


Fig. 2. Median days closed epics spent in pre-development (analysis and refinement) per anonymized project, January–September 2025 vs. 2026 (left), and the median across all 25 projects (right).

B. RQ1: Where Does the Binding Constraint Lie?

Phase 1 identifies the constraints practitioners describe around this acceleration, many of which are binding; Phase 2 explores their anatomy. We organize our insights along the value stream: upstream of implementation (intent, specification, context), lateral to it (cross-team and system boundaries), downstream of it (assurance and release capacity), and above it (the machinery that selects, authorizes, and protects work).

1) Upstream: intent, specification, and usable context: When implementation accelerates, the quality of what flows *into* implementation becomes more consequential. Practitioners locate a first constraint upstream, in the formation of intent and its expression as specifications and usable context, illustrated as issues of delay, clarity and churn: "The business team takes a considerable amount of time to provide the required specifications, and once they do, the requirements tend to change frequently... often resulting in rework and delays" (DX0103).

Notably, the underlying weakness is described as long-standing rather than new: "Accurate and in-depth requirements have always been a problem... we still have poor input poor output" (P4). Nor is the constraint only one of quality — intent formation has its own cadence, which accelerated implementation does nothing to compress: "it ends up taking weeks and weeks, if not longer, for us to get to a point where we can give that to the development team" (P1).

Key observation. Median pre-development time for closed epics fell only 5.9% (22.0 to 20.7 days; 11 of 25 projects increased), while weekly PR/MRs have risen approximately 4-fold since January 2025: implementation has accelerated around an intent-formation cadence that has barely moved.

The net effect is a reweighting of the value stream that practitioners themselves articulate: "the cost of building software is no longer the barrier so it is more about what to build how to build and when to build" (DX0913).

A senior leader identifies a further, somewhat paradoxical, upstream mechanism, wherein the method intended to strengthen intent can, as enacted, separate those who form it from those who realize it: "[developers] were always there, but SDD caused them to be kicked out of the early part of the process" (P12), where clarification had previously occurred informally — "we would just work things back and forth

with the product managers" (P12). The consequence is framed starkly: "The biggest failure that we have is trying to keep developers divested from what the actual vision is" (P12), the respondent highlighting the forced separation of duties across intent formation and implementation. Usable context is further limited by knowledge that no specification yet captures: "Some of that knowledge is in people's heads... in theory, specs is going to try and get that out of people's heads, but... It's hard" (P12).

2) Lateral: cross-team dependency and system boundaries — the dominant theme: Cross-team coordination and dependencies constitute the single largest sub-component in the survey corpus (n=113; Table II), and the interviews supply the mechanism behind that prevalence. One respondent reports that "we can complete small, incremental changes quickly but are then held up releasing due to delays and issues in [team name redacted]" (DX0353), whilst others describe unblocking as escalation-dependent: "This often blocks us completely, or slows us down until someone escalates to get what we need" (DX0114). Some teams structure their work to minimize exposure to boundaries they do not control: "try to reduce our reliances on other teams as much as possible, so that way we control a lot of our own destiny" (P9).

Another respondent's account captures the asymmetry between accelerated implementation and boundary-bound delivery in concrete terms: "He was able to write the code for that in a couple of days, but we're almost a month later and we still don't have that ready to actually deploy... [team name redacted] are saying that they have all these services and platforms available, but then when you actually start to try and use them, okay, its not working" (P3). The code was ready in days; the external team it depended on was not. The binding constraint sits at the organizational boundary, outside both the accelerated activity and the team's control.

3) Downstream: assurance and release capacity as the control point: As implementation accelerates, the emphasis in practitioner accounts shift from writing code to establishing that the code is correct. Interviewees describe output that is plausible in form but wrong in substance, discovered only through active inspection: "It was working on unit tests for me, but... It actually wasn't. It was just going around in circles trying to solve the test" (P8). Survey respondents make the scaling relationship explicit — verification effort grows with the size of the generated change: "the larger the output, the more likely something significant is wrong and needs more effort" (DX0010). Verification thus becomes the load-bearing activity of AI-assisted development, and it is here that the downstream constraint surfaces.

Respondents also highlight an increasing dependency on others to verify AI-assisted output — "we're getting more and more dependent on [member of validation team] and the testing team as well, making sure it's okay" (P8) — against review capacity that is already stretched: "my teams are so understaffed and fragmented between tasks that code reviews are a significant bottleneck" (DX0654). The volume and velocity of change also erode familiarity with the system itself: "we don't know for sure what exactly all of the specific use cases are... and if there are maybe an upstream or downstream impact" (P1).

The downstream constraint appears most acute where verification must span products rather than components. One leader describes a defect in a cross-product flow that was discovered only in production: "this has been broken actually for quite a long time. Nobody caught it... So we can generate all this code... but we still don't have the automated ecosystem tests" (P12).

4) Above: prioritization, decision rights, and capacity protection: Prioritization, and regularly evolving priorities, is referenced as a significant source of frustration by respondents. P10 references the frequent significant changes to deliverables due to changes in business strategy and priority, whilst a survey respondent highlights how competing priorities impede cross-team collaboration: "Cross-team interaction was always a challenge primarily because of competitive priorities for handling intra-team tasks" (DX0326). Another prevalent theme relates to disruptions caused by tool-chain churn and migrations. Respondents attribute these to decisions taken by upstream teams on tools and procedures, each of which may require work of every downstream team, accumulating into what one respondent describes as "relentless" (P3) churn and toil. Corporate-mandated IT and security controls on the PCs assigned to developers (n=107) constitute one of the most prevalent codes in the dataset and represent an example of inhibiting factors explicitly outside of the control of the teams they constrain.

5) The constraint binds heterogeneously: a contrasting case: Here, we draw on survey responses linked to one specific team, alongside interviews with two of its members. This team has long espoused the practices of agile, DevOps and related concepts: "we have continuous delivery to production, which means the team owns the process end-to-end... The developers have a good process for test driven development, and testing is continuous" (P2). Intent is formed collaboratively: "we have a very strong working relationship between BSA, PDM, and engineering. Requirements are clear when we start... If questions arise during development, they are answered or de-scoped quickly" (P2). The team attributes this to a deliberate choice: "one thing we decided years ago was to put all our focus on continuous delivery. If you achieve that, everything else follows" (P2).

Despite these continuity-aligned practices, which, from our analysis, appear considerably more advanced than those of most other teams inspected, this team experiences many of the same binding constraints as others. Whilst references to themes such as requirement clarity, prioritization and verification are significantly less pervasive than in the general population, external dependencies equally impact the team: "Security overhead like compliance and endless reviews cause so much churn... all these processes are coming from further up and accumulate at a team level" (DX0748); "it has already taken months in the process to have an application made available" (DX0789).

6) Answering RQ1: Acceleration concentrated in implementation does not remove the binding constraint on delivery; the constraint resides in the organizational seams surrounding implementation, and it binds heterogeneously. In our case, the constraint re-emerges *upstream*, in intent formation and specification quality (requirements churn; multi-week pre-engineering negotiation); *laterally*, at cross-team and platform boundaries (the largest single category in the survey corpus, n=113); *downstream*, in review, validation and release capacity; and *above*, in the prioritization and decision machinery (priority and migration churn). The mechanism, supplied by the interviews, is a socio-technical rate mismatch: generated volume and implementation speed rise, whilst the cadences of clarification, coordination, verification and authorization remain unchanged, as repository data corroborate (Section V-B1). Which seam binds first is a function of team configuration and product architecture; even a locally continuous team stalls at the first seam it does not own (Section V-B5).

C. RQ2: What Adaptations Would Restore Continuity?

1) Requirements More Critical Than Ever: Respondents converge on the specification as the artifact on which quality is now decided — and the review obligation moves with it: "I don't know if you're going to be able to review all the code, but you have to review your spec" (P5). Proceeding on an unclear specification has concrete consequences: "unless that specification is perfectly clear from the start, the development team are building things maybe customers don't want" (P4). The weakness is long-standing, but its stakes have risen: "we have always struggled with accurate and well prepared requirements and now that's a lot more important again with AI driven development" (P3). One respondent collapses the point into a definition: "Spec is nothing but a glorified prompt" (P10).

Most strikingly, some practitioners frame the shift as a partial inversion of the agile philosophy, relocating effort back toward up-front articulation of intent: "with agile, it is almost the direct opposite of agentic development, because all the work is in the spec" (P5). A second senior leader separately reaches the same diagnosis — "what SDD has taken us back to is waterfall" (P12) — but locates the problem less in up-front specification *per se* than in who participates in it. Recalling an earlier, non-Dell program in which exhaustive up-front specification produced a clean implementation, the participant attributes its success to the fact that "the developers were participating in that up front" (P12). The remedy proposed is co-authorship in small increments: "develop the specs together, and they make sure that they continue to keep them small, right, then you can build up the system gradually" (P12).

2) Extending Development Across the Business: Asked what must change for AI-assisted development to deliver value, practitioners point first not at engineering practices but at the surrounding business. Where development accelerates and adjacent functions do not, the system moves at the pace of its slowest participant: "All four wheels of the car are running in different speed, therefore we are not moving" (P4). The misalignment extends beyond timing to decision-making: "If you have development teams working in an agile way, but the people that are required to make decisions about what those teams are doing... if that's not agile, then the whole thing... falls apart" (P4). One survey respondent anticipates that AI-assisted and Specification-Driven Development "will reduce manual effort but increase the need for clarity in processes, alignment across teams, and discipline in documentation and communication" (DX0909).

Others highlight the need to ensure accelerated effort is focused on the right priorities: "As we progress to build it faster, we need to make sure we're building whats relevant for the business and users" (DX0105) elsewhere, "Priorities are changed very often, and it doesn't feel like the work is coming from what the customers expect" (DX0450). The adaptation practitioners describe is therefore not a further optimization of development teams, but an extension of agile and continuous ways of working into the business functions that feed and govern them.

3) The Need for Continuous Validation: A topic frequently raised by respondents is a perceived heightened need for validation and testing in the AI era, driven by the velocity and volume of generated code: "organizations will need stronger validation frameworks, better specifications, and a cultural shift toward outcome ownership and continuous verification" (DX0910); "we tend to rely too much on AI and forget to verify whether what it gives us is correct or hallucination" (DX0916). A senior leader notes that AI has also enabled previously unattainable levels of continuous validation — "we're developing an agent that says what functional tests should run on the generated code" (P5) — whilst cautioning that replicating customer configurations and scale remains a challenge in which human intuition and experience are crucial. Another senior leader proposes a re-sequencing of SDD itself: "we generate the specs and then we generate the code. Where we find gaps are in the seams... we should be generating specs and then generating the test" (P12), above all "the test that tests at the seams" (P12). The rationale is partly one of human comprehension — "the tests are way easier to look [at] to understand the logic than in the code itself" (P12) — and partly methodological: "we also have abandoned test-driven development with spec-driven development, and I think that's the wrong approach" (P12).

4) Rethinking Roles for the AI Era: The adaptations above imply a redefinition of the roles that must enact them, and practitioners describe that redefinition as already under way: "We are, I think, probably moving away from the term developer" (P5). For some, the shift is unsettling: "Some software engineers are feeling a kind of existential crisis. Software engineers don't want to be curators of markdown" (DX0912). The locus of professional value is moving from the production of code toward problem formulation, judgment, and ownership of outcomes, collapsing previously standalone roles into that of the engineer: "the highest-value skills become: System thinking, Architecture, Domain expertise, Risk analysis, Validation

and review, Specification authoring... Writing code doesn't disappear, but it becomes a smaller percentage of the job" (DX0917). Human comprehension, not code production, is now the scarce resource in the loop: "you have to put your effort to understand what exactly the code is doing... human being is the bottleneck in this new world" (P10). Practitioners also describe role redesign *across* the engineering boundary. Invited to compare the present moment with DevOps, a senior leader endorsed the analogy: "just like we move developers the other way... now we need maybe product managers and developers to move the other way, right? To blur those lines on the other side" (P12) — a blurring rather than a merger, since "Product managers aren't going to become developers... but they do need to include the developers in the development of the requirements" (P12). Locally, a product stakeholder is already "effectively an extension of the engineering team" (P7). These accounts therefore appear to position role redesign not as a consequence of adoption but as a precondition for realizing its value.

5) Answering RQ2: Practitioners indicate the necessary adaptations lie almost entirely outside the implementation phase: (1) re-investment in upstream intent, with the specification *co-authored* by product and engineering in small increments and admitted through readiness checks; (2) extension of agile and continuous ways of working into the business functions that form intent, make decisions and own priorities; (3) continuous, resourced validation scaled to generated volume, including tests derived from the specification — particularly at system seams — before code is generated; and (4) role redesign that shifts engineering value toward specification, judgment and verification, and blurs, without collapsing, the product–engineering boundary. These adaptations align closely with the established CSD and BizDev agenda — practitioners are, in effect, asking their organization to become continuous, with one respondent endorsing the framing of the required change as the upstream counterpart to DevOps. We synthesize them as four continuity gaps in Section VI-C.

VI. Discussion

Our findings re-frame the promise of AI-assisted development as a problem of joint optimization rather than one of tooling. Viewed through the socio-technical lens adopted in Section II-E [32], the organization has applied a step-change in capability to one part of the technical subsystem — implementation — while the social subsystem through which intent is formed, work is coordinated, output is assured, and decisions are made continues to operate at its established cadence. The result is the continuity asymmetry at the center of this paper.

A. The Continuity Asymmetry, Refined: Heterogeneous Binding Constraints and the Seam Argument

A cursory reading of our findings might suggest that AI-driven acceleration has simply moved the constraint to the next slowest stage in the process. Our data suggest something more structural, in that implementation has been accelerated within a system whose remaining loops — intent formation, cross-team coordination, assurance, and authorization — are rate-limited by human cadence rather than by tooling. Repository-level evidence of diminishing returns [6] suggests that further investment in the accelerated subsystem is unlikely by itself to outrun such a constraint. Crucially, the resulting constraint does not bind in one universal place (Section V-B6). This heterogeneity refines the finding of Tkalich et al. that dependencies between software teams and support functions dominate friction in large-scale continuous software engineering [10]: in an AI-accelerated setting, *which* dependency binds is a function of team configuration and product architecture, and no single organization-wide remedy addresses every team. Diagnosis must therefore precede intervention.

Our contrasting case (Section V-B5) underpins what we term the *seam argument*. That team has locally closed the loops others struggle with yet still stalls at the processes it does not own. Local continuity, in other words, raises a team's ceiling only as far as the first organizational seam it does not control. Continuity is therefore an enterprise property: two decades of agile and DevOps practice have optimized flow *inside* the team boundary, leaving the boundaries themselves dominant — a condition acceleration renders more costly. This paradigm aligns with the BizDev agenda of extending continuity beyond development and operations into the business functions that form intent, authorize work, and consume feedback [15], [13], which we develop in Section VI-B. What the AI era changes is not the direction of that agenda but its urgency. These findings also offer a mechanism-level account of the phenomenon described by Bosch and Holmström Olsson, in which organizations automate separate tasks without reinventing the end-to-end engineering cycle [29].

B. From DevOps to BizDev: Blurring the Product–Engineering Seam

The seam argument raises the question of which boundaries an organization should address first, and our data suggest a historical symmetry. DevOps addressed a downstream seam, between development and operations, by blurring role boundaries and sharing ownership so that code could flow continuously to production; the team in our contrasting case has largely closed that seam locally (Section V-B5). AI-assisted implementation now places the greatest pressure on the opposite, upstream seam — between those who form intent and those who realize it. Invited to draw the comparison, one senior leader endorsed Fitzgerald and Stol's framing of BizDev as the business-facing counterpart to DevOps [15], [13] — "just the other side of the equation" (P12).

Our evidence refines that agenda in three ways. First, practitioners describe a *blurring* rather than a *collapse* of roles: product managers are not expected to become developers, but developers are expected to participate in forming intent (P12; Section V-C). Second, SDD is not inherently continuous: depending on how the specification is produced, it may re-create the hand-off that agile and DevOps sought to remove — what participants describe as a return to "waterfall" (P5, P12).

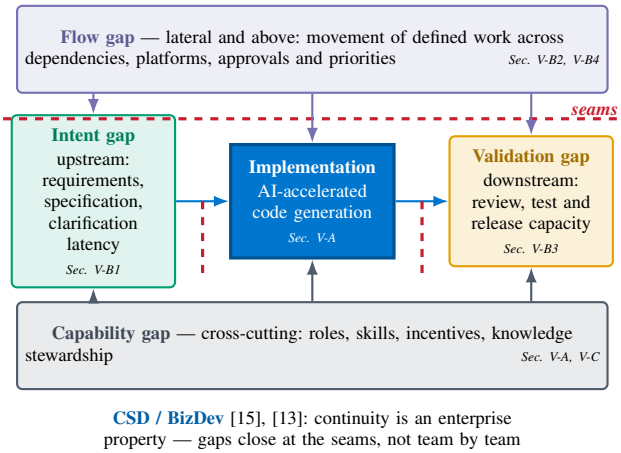


Fig. 3. The four continuity gaps. Dashed lines mark organizational seams bounding AI-accelerated implementation; following practitioner feedback (P12), the flow gap sits above the value stream, bounding implementation rather than feeding into it.

Third, the blurring extends to validation. Deriving tests from the specification, particularly at system seams, renders intent — otherwise far harder to verify than generated output [27] — partly verifiable through executable feedback, narrowing the asymmetry between the intent and validation gaps (Section VI-C) and restoring the test-first element of the original formulation of SDD [34].

C. Four Continuity Gaps: CSD as the Organizing Response

We synthesize our findings as a theoretical contribution into four **continuity gaps**: junctions of incomplete joint optimization at which organizational flow breaks, and for which CSD already provides the organizing concept. Figure 3 locates each gap in the delivery system, and Table III operationalizes them as a practitioner diagnostic; across all four gaps, we propose, as an overarching premise, a DevOps-style blurring of lines between product management, business analysis and development.

Intent gap: a discontinuity between those who decide what is to be built, and why, and those who build it; it opens when intent reaches implementation late, incomplete or unstable.

Flow gap: a discontinuity in the movement of work already defined across team, platform and governance boundaries; it opens when ready work waits on another team, an approval or a reprioritization outside the team's control.

Validation gap: a discontinuity between producing a change and establishing that it is correct and releasable; it opens when review, test and environment capacity cannot keep pace with the volume and size of change.

Capability gap: a cross-cutting shortfall in the roles, skills, incentives and shared knowledge required to close the other three.

Intent and flow gaps are the most easily confused, as both present as waiting; they are distinguished by what the work waits *for* — clarity on what to build (intent), or the capacity and permission to proceed with work already defined (flow).

Stated as our position: **the most successful organizations of the AI era will not necessarily be those with the greatest technical resources, but those structured for continuity across these gaps. As continuity is an enterprise property bounded by organizational seams, that structure cannot be addressed team by team.**

Importantly, the gaps are not a product of AI. Each corresponds to a disconnect documented before AI-assisted development — between business strategy and development [13], across dependencies between teams and support functions [10], and across the organizational, platform and cultural dimensions of CSD transformation [17] — and practitioners describe the underlying weaknesses as long-standing (P3, P4). Our contrasting case, whose continuity practices predate AI adoption, stalls at seams unrelated to AI, namely security review and application provisioning (Section V-B5). What AI changes is the cost of the gaps rather than their existence: an implementation phase that once absorbed, and so concealed, upstream and downstream latency no longer does so. Continuity asymmetry is therefore the AI-era expression of the gaps, not their cause, and the diagnostic applies irrespective of AI adoption.

D. Mitigations in Flight

We distinguish team-level from enterprise-level mitigations, as the distinction bears directly on the seam argument (Section VI-A).

Team-level practices. Against the intent and validation gaps, teams report readiness-checked intake (P2), product stakeholders embedded in engineering (P7), a long-standing "focus on small, iterative pull requests" (P7), continuous automated testing (P2), tiered functional testing on the main branch, and AI-assisted test selection (P5).

Enterprise-level initiatives. Organization-wide interventions evident in the data include changes to token economics (DX0007), changes to organizational and team-level structures to support SDD (P5), and the SDD transition itself, a cross-functional intervention described as a methodology wherein "humans define intent, constraints, and outcomes while AI executes much of the implementation" (DX0917). However, beyond SDD and pod-based structures, which act principally on the intent gap, none of our evidence describes an enterprise-level redesign of the dependency, governance or decision machinery that sits between and above teams.

E. Comparison to Prior Work

Our findings converge with recent repository-level evidence. He et al. state that "deliberate process adaptations—those that scale quality assurance with AI-era velocity—are necessary to realize sustained benefits" [5]. Our validation gap reaches the same conclusion from within an enterprise, but locates the mechanism organizationally, in review, test and release capacity, rather than in the accumulation of technical debt; both accounts suggest that assurance must be designed as a first-class capacity rather than assumed.

TABLE III
DIAGNOSING CONTINUITY GAPS IN AGENTIC-ERA DELIVERY ORGANIZATIONS (n = SUB-COMPONENT CODE COUNT; † = OBSERVED INTERVENTION).

Gap	Observable symptoms	Candidate measures	Potential interventions
Intent	Vague, churning requirements (n=47); waiting on clarification; late customer feedback (n=22); developers excluded from intent formation	Clarification latency; requirement readiness at sprint start; % work admitted via readiness check	Readiness-checked intake, answer-or-de-scope clarification† (P2); co-authored, small-batch specs (PDM + engineering) without SDD hand-off
Flow	Dependency (n=113) and approval (n=45) queues; delayed deploys; escalations; capacity lost to injected work	Cross-team wait states; approval altitude; deploy independence; % roadmap capacity protected	Team autonomy and dependency redesign [42], [43]; pod-based work structures; low-dependency architectural concepts
Validation	Saturated review/test capacity (n=37); large PRs; environment instability; no automated cross-product tests	Review latency; escaped defects; verification coverage of AI output; seam/ecosystem test coverage	Small batches† (P7); tiered continuous testing† (P2, P5); AI-assisted test selection† (P5); spec→test→code incl. seam tests; validation harnesses [33]
Capability (cross-cutting)	Fragmented knowledge and documentation gaps (n=109); developer identity strain; silo/inter-team competition	Specification/validation skill coverage; ownership [44]; documentation coverage; assurance workload	Role redesign; AI rules and skills for documentation generation; assurance recognized as capacity
	Overarching intervention (all gaps): DevOps-style blurring of roles across the software delivery lifecycle (Section VI-B)		

VII. THREATS TO VALIDITY

We follow the case-study validity categories of Runeson and Höst [36]. **External validity.** As a single-company study, our findings are not statistically generalizable; rather, we argue for transferability to large, traditionally structured enterprises with heterogeneous software domains and scaled agile operating models, supported by the close correspondence between our themes and documented large-scale frictions [10], [45], [46], [47], [48]. **Construct validity.** Open-text survey comments are self-selected and may over-represent grievances. The Phase 1 unit of analysis is the comment, not the respondent, so prevalence counts are not respondent proportions. The interview sample (n=12) is purposive. Convergence between independently collected survey and interview evidence mitigates these method-specific effects. The DevOps analogy reported in Section V-C was introduced by the interviewer and endorsed and elaborated by the participant, who noted not having previously considered it (P12). We therefore report it as practitioner endorsement of a researcher-proposed concept, its corroboration resting on other accounts in the dataset (P2, P4, P7). **Internal validity.** We cannot exclude the rival explanation that AI has made existing constraints more *visible* rather than *worse*; our case for continuity holds under either. To support the acceleration premise (Section V-A) and the upstream asymmetry (Section V-B1), we include internal data on PR/MR volume and size, stories completed, and Jira epic status histories. The PR/MR and story data measure implementation output rather than delivery, and we do not attribute their growth to AI assistance alone; other factors such as changes in repository coverage may also contribute. All these data are drawn from pre-existing internal reporting rather than collected for this study, and are susceptible to distortion from Jira workflow changes, repository migrations and process changes; we therefore treat them as descriptive corroboration of practitioner accounts, not as statistically significant evidence. **Reliability.** Provisional coding was AI-assisted under the controls described in Section IV-D. Researcher employment at the studied company introduces insider bias and power effects on disclosure, while enabling access and interpretive depth. Showing the emerging model toward the end of the interview to one participant (P12) may have shaped responses toward it.

VIII. CONCLUSION

The findings from our case study further the growing recognition that adoption of AI-assisted development tools alone is inadequate to transform product development. Our study indicates that a lack of joint optimization in socio-technical constructs inhibits the organization from realizing the true value of AI. The continuity gaps themselves are not a product of AI. Misaligned socio-technical design has long been the inhibiting factor in large-scale software development; however, we argue this is now more critical than ever. Drawing on practitioner accounts, we suggest that just as DevOps blurred the boundary between development and operations, the AI era calls for a comparable blurring between product and engineering — the BizDev agenda, whose urgency AI-assisted development has markedly increased. We therefore call for a renewed focus on CSD in large-scale contexts, as the organizing logic for the AI era.

ACKNOWLEDGMENT

Thank you to Donal Golden and Bob MacFarlane (Dell Technologies) for their support and invaluable guidance.

REFERENCES

[1] M. Watanabe, H. Li, Y. Kashiwa, B. Reid, H. Iida, and A. E. Hassan, "On the use of agentic coding: An empirical study of pull requests on GitHub," *ACM Trans. Softw. Eng. Methodol.*, 2026, doi: 10.1145/3798166.

[2] K. Mao, T. Kapus, C. T. Åhs, M. Marescotti, D. Ip, Á. Hajdu, S. Cela, and A. Banerjee, "WhatsCode: Large-scale GenAI deployment for developer efficiency at WhatsApp," in *Proc. IEEE/ACM 48th Int. Conf. Softw. Eng.: Softw. Eng. in Practice (ICSE-SEIP)*, 2026, doi: 10.1145/3786583.3786877.

[3] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, "The impact of AI on developer productivity: Evidence from GitHub Copilot," 2023, *arXiv:2302.06590*.

[4] R. Robbes, T. Matricon, T. Degueule, A. Hora, and S. Zacchiroli, "Agentic much? Adoption of coding agents on GitHub," *ACM Trans. Softw. Eng. Methodol.*, 2026, doi: 10.1145/3822180.

[5] H. He, C. Miller, S. Agarwal, C. Kästner, and B. Vasilescu, "Speed at the cost of quality: How Cursor AI increases short-term velocity and long-term complexity in open-source projects," in *Proc. 23rd Int. Conf. Mining Softw. Repositories (MSR)*, 2026, doi: 10.1145/3793302.3793349.

[6] S. Agarwal, H. He, and B. Vasilescu, "AI IDEs or autonomous agents? Measuring the impact of coding agents on software development," in *Proc. 23rd Int. Conf. Mining Softw. Repositories (MSR)*, 2026, doi: 10.1145/3793302.3793589.

[7] M.-A. Storey, R. Hoda, A. Maciel Paz Milani, and M. T. Baldassarre, "Guiding principles for mixed methods research in software engineering," *Empir. Softw. Eng.*, vol. 30, no. 5, Art. no. 138, 2025.

[8] E. Klotins and E. Talbert-Goldstein, "Organizational conflicts in the adoption of continuous software engineering," in *Agile Processes in Software Engineering and Extreme Programming (XP 2023)*, LNBIP vol. 475. Cham, Switzerland: Springer, 2023, pp. 149–164, doi: 10.1007/978-3-031-33976-9_10.

[9] E. Klotins and T. Gorschek, "Continuous software engineering in the wild," in *International Conference on Software Quality*, pp. 3–12, 2022, *arXiv:2203.12409*.

[10] A. Tkalich, E. Klotins, and N. B. Moe, "Identifying critical dependencies in large-scale continuous software engineering," in *Proc. 29th Int. Conf. Evaluation and Assessment in Softw. Eng. (EASE)*, 2025, doi: 10.1145/3756681.3757027.

[11] S. H. Appelbaum, "Socio-technical systems theory: An intervention strategy for organizational development," *Manage. Decis.*, vol. 35, no. 6, pp. 452–463, 1997, doi: 10.1108/00251749710173823.

[12] T. Theunissen and U. van Heesch, "Specification in continuous software development," in *Proc. 22nd Eur. Conf. Pattern Languages of Programs (EuroPLoP)*, 2017, doi: 10.1145/3147704.3147709.

[13] B. Fitzgerald and K.-J. Stol, "Continuous software engineering: A roadmap and agenda," *J. Syst. Softw.*, vol. 123, pp. 176–189, 2017, doi: 10.1016/j.jss.2015.06.063.

[14] D. Dennehy and K. Conboy, "Going with the flow: An activity theory analysis of flow techniques in software development," *J. Syst. Softw.*, vol. 133, pp. 160–173, 2017, doi: 10.1016/j.jss.2017.01.007.

[15] B. Fitzgerald and K.-J. Stol, "Continuous software engineering and beyond: Trends and challenges," in *Proc. 1st Int. Workshop Rapid Continuous Softw. Eng. (RCoSE)*, 2014, pp. 1–9, doi: 10.1145/2593812.2593813.

[16] E. Klotins and T. Gorschek, "Continuous software engineering: Introducing an industry readiness model," *IEEE Softw.*, vol. 40, no. 4, pp. 77–87, 2023.

[17] A. Barbala, R. Ulfsnes, N. B. Moe, N. H. Andersen, and J. Emmerhoff, "Large-scale continuous software engineering: Exploring three dimensions of transformation," in *Proc. 29th Int. Conf. Evaluation and Assessment in Softw. Eng. (EASE)*, 2025, doi: 10.1145/3756681.3756962.

[18] H. Edison, X. Wang, and K. Conboy, "Comparing methods for large-scale agile software development: A systematic literature review," *IEEE Trans. Softw. Eng.*, vol. 48, no. 8, pp. 2709–2731, 2022, doi: 10.1109/TSE.2021.3069039.

[19] T. Dingsøyr, N. B. Moe, T. E. Fægri, and E. A. Seim, "Exploring software development at the very large-scale: A revelatory case study and research agenda for agile method adaptation," *Empir. Softw. Eng.*, vol. 23, no. 1, pp. 490–520, 2018, doi: 10.1007/s10664-017-9524-2.

[20] K. Dikert, M. Paasivaara, and C. Lassenius, "Challenges and success factors for large-scale agile transformations: A systematic literature review," *J. Syst. Softw.*, vol. 119, pp. 87–108, 2016, doi: 10.1016/j.jss.2016.06.013.

[21] J. Bosch and H. Holmström Olsson, "The end of agile as we know it," 2024, *SSRN preprint 5008903*.

[22] S. Abrahão, J. Grundy, M. Pezzè, M.-A. Storey, and D. A. Tamburri, "Software engineering by and for humans in an AI era," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, pp. 1–46, 2025, doi: 10.1145/3715111.

[23] R. Hoda, "Toward agentic software engineering beyond code: Framing vision, values, and vocabulary," in *Proc. 2026 Int. Workshop Agentic Eng. (AGENT)*, 2026, doi: 10.1145/3786167.3788422.

[24] Y. Lyu, Z. Yang, J. Shi, J. Chang, Y. Liu, and D. Lo, "'My productivity is boosted, but …' Demystifying users' perception on AI coding assistants," in *Proc. 40th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, pp. 191–203, 2025, doi: 10.1109/ASE63991.2025.00024.

[25] R. Choudhuri, C. Badea, C. Bird, J. L. Butler, R. DeLine, and B. Houck, "AI where it matters: Where, why, and how developers want AI support in daily work," in *Proc. IEEE/ACM 48th Int. Conf. Softw. Eng.: Softw. Eng. in Practice (ICSE-SEIP)*, 2026, doi: 10.1145/3786583.3786885.

[26] U. Cihan, V. Haratian, A. İçöz, M. K. Gül, Ö. Devran, E. F. Bayendur, B. M. Uçar, and E. Tüzün, "Automated code review in practice," in *Proc. IEEE/ACM 47th Int. Conf. Softw. Eng.: Softw. Eng. in Practice (ICSE-SEIP)*, 2025, pp. 425–436, doi: 10.1109/ICSE-SEIP66354.2025.00043.

[27] S. Alvanakis Apostolou, J. Bosch, and H. Holmström Olsson, "Assistance to autonomy: A systematic literature review of agentic AI across the software development life cycle," 2026, *arXiv:2605.15245*.

[28] V. Saklamaeva and L. Pavlič, "The potential of AI-driven assistants in scaled agile software development," *Appl. Sci.*, vol. 14, no. 1, p. 319, 2024, doi: 10.3390/app14010319.

[29] J. Bosch and H. Holmström Olsson, "Towards AI-driven organizations," in *Proc. Euromicro Conf. Softw. Eng. and Advanced Applications (SEAA 2025)*, LNCS vol. 16083. Cham, Switzerland: Springer, 2026, pp. 280–295, doi: 10.1007/978-3-032-04207-1_19.

[30] M. E. Conway, "How do committees invent?" *Datamation*, vol. 14, no. 4, pp. 28–31, 1968.

[31] O. Elazhary, C. Werner, Z. S. Li, D. Lowlind, N. A. Ernst, and M.-A. Storey, "Uncovering the benefits and challenges of continuous integration practices," *IEEE Trans. Softw. Eng.*, vol. 48, no. 7, pp. 2570–2583, 2022, doi: 10.1109/TSE.2021.3064953.

[32] M. C. Davis, R. Challenger, D. N. W. Jayewardene, and C. W. Clegg, "Advancing socio-technical systems thinking: A call for bravery," *Appl. Ergon.*, vol. 45, no. 2, pp. 171–180, 2014, doi: 10.1016/j.apergo.2013.02.009.

[33] P. Mandl and P. Mandl, "AI-driven software development: A pragmatic path to agentic development processes," 2026, *arXiv:2606.15283*.

[34] J. S. Ostroff, D. Makalsky, and R. F. Paige, "Agile specification-driven development," in *Extreme Programming and Agile Processes in Software Engineering (XP 2004)*, LNCS vol. 3092. Berlin, Germany: Springer, 2004, pp. 104–112, doi: 10.1007/978-3-540-24853-8_12.

[35] D. B. Piskala, "Spec-driven development: From code to contract in the age of AI coding assistants," 2026, *arXiv:2602.00180*.

[36] P. Runeson and M. Höst, "Guidelines for conducting and reporting case study research in software engineering," *Empir. Softw. Eng.*, vol. 14, no. 2, pp. 131–164, 2009, doi: 10.1007/s10664-008-9102-8.

[37] V. Braun and V. Clarke, "Using thematic analysis in psychology," *Qual. Res. Psychol.*, vol. 3, no. 2, pp. 77–101, 2006, doi: 10.1191/1478088706qp063oa.

[38] V. Clarke and V. Braun, "Thematic analysis," *J. Posit. Psychol.*, vol. 12, no. 3, pp. 297–298, 2017, doi: 10.1080/17439760.2016.1262613.

[39] V. Braun and V. Clarke, "One size fits all? What counts as quality practice in (reflexive) thematic analysis?" *Qual. Res. Psychol.*, vol. 18, no. 3, pp. 328–352, 2021, doi: 10.1080/14780887.2020.1769238.

[40] Anthropic, "Claude Opus 5.5," 2026. [Online]. Available: https://www.anthropic.com/claude-opus-5-5

[41] Association for Computing Machinery, "ACM policy on authorship: Frequently asked questions," 2026. [Online]. Available: https://www.acm.org/publications/policies/frequently-asked-questions

[42] H. Holmström Olsson, H. Alahyari, and J. Bosch, "Climbing the 'Stairway to Heaven'—A multiple-case study exploring barriers in the transition from agile development towards continuous deployment of software," in *Proc. 38th Euromicro Conf. Softw. Eng. and Advanced Applications (SEAA)*, 2012, pp. 392–399, doi: 10.1109/SEAA.2012.54.

[43] T. Savor, M. Douglas, M. Gentili, L. Williams, K. Beck, and M. Stumm, "Continuous deployment at Facebook and OANDA," in *Proc. 38th Int. Conf. Softw. Eng. Companion (ICSE-C)*, 2016, pp. 21–30, doi: 10.1145/2889160.2889223.

[44] V. Chen, J. He, B. Williams, J. Valentino, and A. Talwalkar, "Beyond the commit: Developer perspectives on productivity with AI coding assistants," in *Proc. IEEE/ACM 48th Int. Conf. Softw. Eng.: Softw. Eng. in Practice (ICSE-SEIP)*, 2026, doi: 10.1145/3786583.3786848.

[45] M. Paasivaara and C. Lassenius, "Challenges and success factors for large-scale agile transformations: A research proposal and a pilot study," in *Proc. Sci. Workshop XP2016*, 2016, doi: 10.1145/2962695.2962704.

[46] M. Paasivaara, C. Lassenius, and V. T. Heikkilä, "Inter-team coordination in large-scale globally distributed Scrum: Do Scrum-of-Scrums really work?" in *Proc. ACM-IEEE Int. Symp. Empirical Softw. Eng. and Measurement (ESEM)*, 2012, pp. 235–238, doi: 10.1145/2372251.2372294.

[47] S. Bick, K. Spohrer, R. Hoda, A. Scheerer, and A. Heinzl, "Coordination challenges in large-scale software development: A case study of planning misalignment in hybrid settings," *IEEE Trans. Softw. Eng.*, vol. 44, no. 10, pp. 932–950, 2018.

[48] E. Knauss, "The missing requirements perspective in large-scale agile system development," *IEEE Softw.*, vol. 36, no. 3, pp. 9–13, 2019, doi: 10.1109/MS.2019.2896708.